



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240216

姓 名： 李晓翠

实验（1）

【实验目的】

1. 掌握并完成 LMStudio 本地大语言模型的部署与配置方法
2. 学会统计 LLM 调用的 token 消耗、响应时间和推理速度等关键指标
3. 不断测试不同参数的大语言模型加载方式，找到最适合自己电脑的配置

【实验内容】

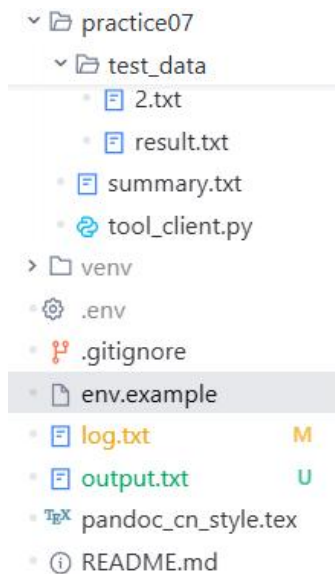
1.1 实验环境准备

下载并安装 LMStudio 后访问 LMStudio 官网(<https://lmstudio.ai/>)
下载对应操作系统的版本，安装完成后加载任意开源大语言模型（如 Qwen2.5、Llama3、DeepSeek 等）。

查看 Python 版本

```
PS D:\tishi> python --version  
Python 3.12.0  
PS D:\tishi> █
```

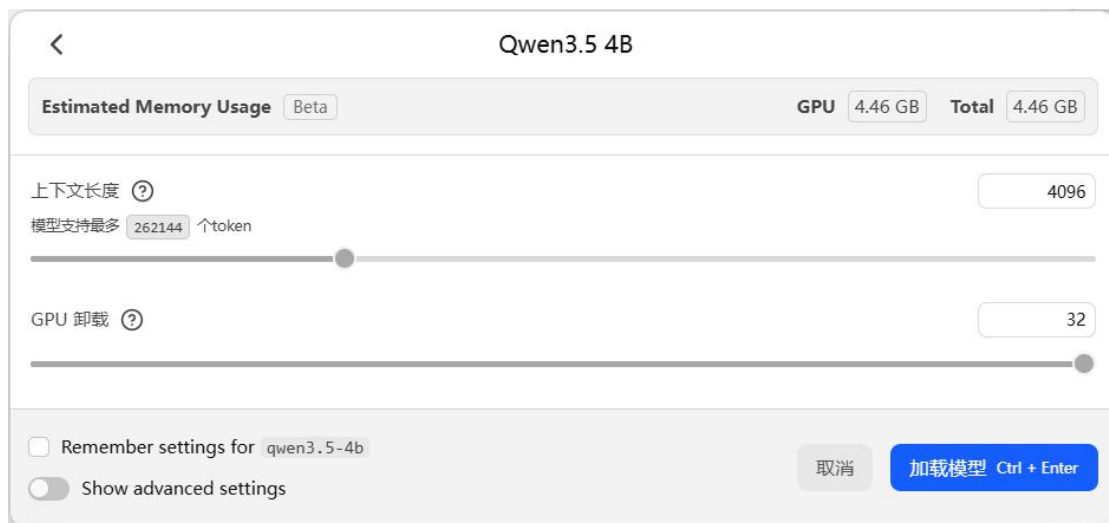
在项目根目录下创建一个 env.example 文件（用户会手动复制一份 .env 文件并填写正确参数），填入项目使用的 OpenAI 兼容协议的 LLM 的 Base_URL、Model、token 等信息的模版。



```
1 # OpenAI兼容协议LLM配置
2 BASE_URL="http://localhost:11434/v1"
3 MODEL="llama3"
4 API_KEY="your-api-key-here"
```

新建 practice01 目录，在其中创建一个 py 文件，读取项目根目录的 .env 文件内容，用 python 标准 http 库访问用户定义的 LLM，并且统计 token 消耗、时间、和 token/s 速度。

加载模型，调整模型参数



给 AI 提问，得出结果



问题 输出 调试控制台 终端

Let's go with the "two-tired" one. It's short and clear.

Wait, let's just pick a very reliable one.

--- Token Statistics ---

响应时间: 13.22 秒

提示词 Token: 20

回复 Token: 500

总 Token: 520

生成速度: 37.83 tokens/秒

=====

PS F:\人工提示\practice>

< Qwen3.5 4B

Estimated Memory Usage Beta GPU 3.81 GB Total 4.51 GB

上下文长度 ? 6217
模型支持最多 262144 个token

GPU 卸载 ? 26

☒ Remember settings for qwen3.5-4b
☐ Show advanced settings

取消 加载模型 Ctrl + Enter

问题 输出 调试控制台 终端

How about this:

Why did the bicycle fall over? Because it was two-tired. (Very short, punny).

Let's go with the

--- Token Statistics ---

响应时间: 21.77 秒

提示词 Token: 20

回复 Token: 500

总 Token: 520

生成速度: 22.97 tokens/秒

=====

PS F:\人工提示\practice>

【实验总结】

通过本次实验，我完成了 AI 智能体开发项目的基础环境搭建，包括



初始化 venv 虚拟环境、配置 Python 开发环境、创建 .env 配置模板，并通过 LMStudio 成功本地部署了 Qwen3.5-4B 模型。在此基础上，我使用 Python 标准 http 库编写了 LLM 调用程序，实现了对 token 消耗、响应时间和推理速度的自动化统计。

为找到最适合本机配置的模型加载参数，使用相同的提示词，测试了两组 GPU 卸载层数配置。当 GPU 卸载层数从 26 层提升至 32 层时，生成速度从 22.97 tokens/秒提升至 37.83 tokens/秒，性能提升约 64.7%，响应时间由 21.77 秒缩短至 13.22 秒。这说明更多的模型层数加载到 GPU 上，能显著减少 CPU 与 GPU 之间的数据搬运开销，充分发挥 GPU 的并行计算优势。同时，适当缩短上下文长度、增加 GPU 卸载层数，是在显存有限情况下提升推理效率的更优策略。综合速度与显存占用，本机的最佳配置为 GPU 卸载 32 层配合上下文长度 4096，此时显存占用约 4.46GB，生成速度达到 37.83 tokens/秒，能够满足日常 AI 应用开发的实时性需求。

实验过程中，模型加载耗时较长是主要困难，初次下载和加载 Qwen3.5-4B 模型时因网络不稳定和模型文件较大花费了大量时间。在寻找适合本机显存的配置时，也曾因 GPU 卸载层数设置过高导致显存溢出，或因设置过低导致推理速度过慢，这使我认识到本地 LLM 部署的核心矛盾是显存容量与推理速度的平衡，需要根据硬件条件反复测试。使用 Python 标准 http 库调用 LLM 时，还需注意 OpenAI 兼容协议的请求格式以及通过响应体提取 token 消耗信息，计时统计需在请求发送前和响应接收后分别记录时间戳。



本次实验不仅让我掌握了 LMStudio 本地部署 LLM 的完整流程，更重要的是建立了配置、测试、数据分析、优化的工程化思维。通过量化对比不同参数下的性能指标，我能够基于数据而非直觉做出配置决策，这为后续学习更复杂的 AI 应用开发打下了坚实的工程实践基础。